

Para Bank Automation Code Structure

Selenium + TestNG + Maven | Page Object Model Framework

Created By: Tapabrata Sikder

Batch: SDET-B3

Report to: Vaishali

Date: 15/06/2026

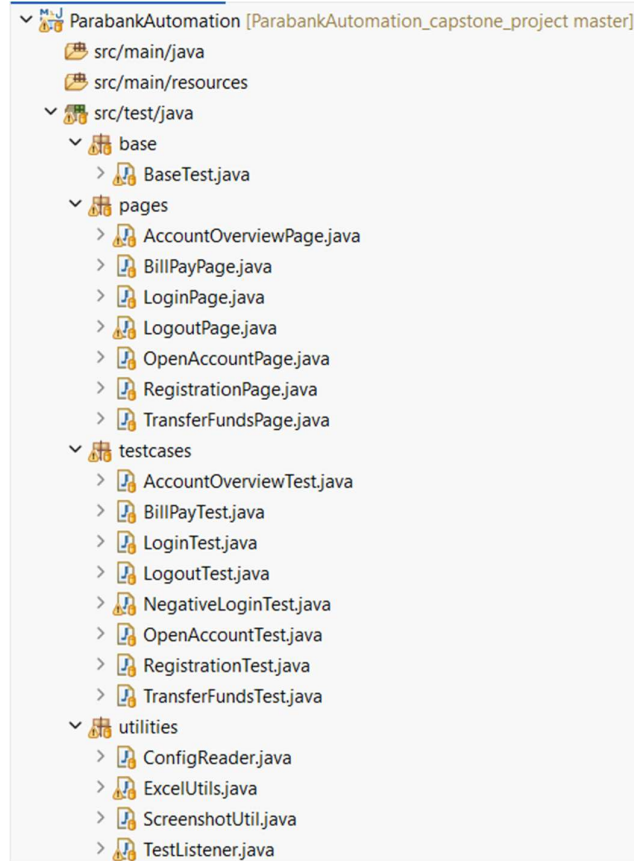
GitHub link: [GitHub](#)

Video Link: [Project Video](#)

Table of Contents

1. Project Structure	3
2. Pom.xml file	3
3. SRC/TEST/JAVA.....	6
I. src/test/java/base/BaseTest.java	6
II. src/test/java/pages	7
i. RegistrationPage.java	7
ii. LoginPage.java	9
iii. OpenAccountPage.java.....	10
iv. AccountOverviewPage.java	11
v. TransferFundsPage.java.....	12
vi. BillPayPage.java.....	13
vii. LogOutPage.java.....	16
III. src/test/java/testcases	17
i. RegistrationTest.java	17
ii. LoginTest.java	18
iii. NegativeLoginTest.java.....	19
iv. OpenAccountTest.java.....	20
v. AccountOverviewTest.java.....	21
vi. TransferFundsTest.java.....	23
vii. BillPayTest.java.....	25
viii. LogOutTest.java	27
IV. src/test/java/utilities	29
i. ConfigReader.java	29
ii. ExcelUtils.java.....	29
iii. ScreenshotUtil.java.....	31
iv. TestListener.java	32
4. Src/test/resources	33
i. Config.properties	33
ii. TestData.xlsx	33
5. Testng.xml	34
6. Console Output.....	35
7. Jenkins file	36

1. Project Structure



2. Pom.xml file

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd>
```

```

<modelVersion>4.0.0</modelVersion>
<groupId>com.capstone</groupId>
<artifactId>ParabankAutomation</artifactId>
<version>0.0.1-SNAPSHOT</version>

<dependencies>
  <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.44.0</version>
  <scope>compile</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.12.0</version>
  <scope>test</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/org.apache.poi/poi-ooxml -->
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.5.1</version>
  <scope>compile</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/org.apache.poi/poi -->
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi</artifactId>
  <version>5.5.1</version>
  <scope>compile</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/io.cucumber/cucumber-java -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.34.3</version>
  <scope>compile</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/io.cucumber/cucumber-testng -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-testng</artifactId>
  <version>7.34.3</version>
  <scope>compile</scope>
</dependency>

<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.9.1</version>
</dependency>

<dependency>

```

```

    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.16.1</version>
</dependency>

<dependency>
  <groupId>org.apache.maven.surefire</groupId>
  <artifactId>surefire-testng</artifactId>
  <version>3.5.2</version>
</dependency>

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.2</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
        <includes>
          <include>/**/*.Test.java</include>
        </includes>
      </configuration>
    </plugin>
  </plugins>
</build>

</project>

```

3. SRC/TEST/JAVA

I. src/test/java/base/BaseTest.java

```
package base;

import io.github.bonigarcia.wdm.WebDriverManager;
import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import utilities.ConfigReader;

@Listeners(utilities.TestListener.class)
public class BaseTest {

    public static WebDriver driver;

    public static String capturedAccountId;

    public static String newAccountId;

    @BeforeMethod
    public void setup() throws Exception {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        String url = ConfigReader.getProperty("appurl");

        driver.get(url);

    }

    public static WebDriver getDriver() {

        return driver;

    }
}
```

```

    @AfterMethod
    public void tearDown() {
        if(driver != null) {
            driver.quit();
        }
    }
}

```

II. src/test/java/pages

i. [RegistrationPage.java](#)

```

package pages;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class RegistrationPage {

    private WebDriver driver;
    private WebDriverWait wait;

    private By regLink = By.linkText("Register");
    private By firstName = By.id("customer.firstName");
    private By lastName = By.id("customer.lastName");
    private By address = By.id("customer.address.street");
    private By city = By.id("customer.address.city");
    private By state = By.id("customer.address.state");
    private By zip = By.id("customer.address.zipCode");
    private By phone = By.id("customer.phoneNumber");
    private By ssnField = By.id("customer.ssn");

```

```

private By username = By.id("customer.username");
private By password = By.id("customer.password");
private By cnfpassword = By.id("repeatedPassword");
private By registerButton = By.xpath("//input[@value='Register']");

public RegistrationPage(WebDriver driver){
    this.driver=driver;
    this.wait=new WebDriverWait(driver, Duration.ofSeconds(10));
}

public void register(String fname, String lname, String adr, String cty, String st, String zp,
String phn, String ssnNum, String user, String pwd, String cnfpwd) {
    wait.until(ExpectedConditions.elementToBeClickable(regLink)).click();

    wait.until(ExpectedConditions.visibilityOfElementLocated(firstName)).sendKeys(fname);
    driver.findElement(laStName).sendKeys(lname);
    driver.findElement(address).sendKeys(adr);
    driver.findElement(city).sendKeys(cty);
    driver.findElement(state).sendKeys(st);
    driver.findElement(zip).sendKeys(zp);
    driver.findElement(phone).sendKeys(phn);
    driver.findElement(ssnField).sendKeys(ssnNum);
    driver.findElement(username).sendKeys(user);
    driver.findElement(password).sendKeys(pwd);
    driver.findElement(cnfpassword).sendKeys(cnfpwd);
    driver.findElement(registerButton).click();
}

public String confirmRegister() {
    return
    wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//h1[@class='title']"))).getText(
    );
}
}

```


ii. LoginPage.java

```
package pages;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class LoginPage {

    private WebDriver driver;
    private WebDriverWait wait;

    By user = By.name("username");
    By pass = By.name("password");
    By submit = By.xpath("//input[@value='Log In']");

    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver=driver; // current class member variable
        // as soon as the LoginPage object created, wait is ready to be used
        this.wait=new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    public void loginUser(String uname, String pwd) {
        wait.until(ExpectedConditions.visibilityOfElementLocated(user)).sendKeys(uname);
        driver.findElement(pass).sendKeys(pwd);
        driver.findElement(submit).click();
    }

    public boolean isLoginErrorDisplayed() {
        return driver.findElements(By.className("error")).size() > 0;
    }
}
```

iii. OpenAccountPage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class OpenAccountPage {

    private WebDriver driver;
    private WebDriverWait wait;

    By openAccountLink = By.linkText("Open New Account");
    By accountTypeSelect = By.id("type");
    By newAccountId = By.id("newAccountId");

    public OpenAccountPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, java.time.Duration.ofSeconds(10));
    }

    public void clickOpenAccount() {
        wait.until(ExpectedConditions.elementToBeClickable(openAccountLink)).click();
    }

    public void openNewAccount(String accountType) {

        wait.until(ExpectedConditions.visibilityOfElementLocated(accountTypeSelect)).sendKeys(accountType);

        Select fromDropdown = new Select(driver.findElement(accountTypeSelect));
        fromDropdown.selectByVisibleText(accountType);
    }
}
```

```

        driver.findElement(By.xpath("//input[@value='Open New Account']")).click();
    }

    public String getNewAccountId() {
        return
        wait.until(ExpectedConditions.visibilityOfElementLocated(newAccountId)).getText();
    }
}

```

iv. AccountOverviewPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedCondition;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class AccountOverviewPage {

    private WebDriver driver;
    private WebDriverWait wait;

    By totalBalance = By.xpath("//td[contains(normalize-space(), 'Total')]/following-sibling::td[1]");

    public AccountOverviewPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, java.time.Duration.ofSeconds(10));
    }

    public String getAccountId(int index) {
        By accountId = By.xpath("//table[@id='accountTable']/tr[" + index + "]/td[1]/a");

```

```

        return wait.until(ExpectedConditions.visibilityOfElementLocated(accountId)).getText();
    }

    public String getBalance(int index) {
        By balance = By.xpath("//table[@id='accountTable']/tr[" + index + "]/td[2]");
        return wait.until(ExpectedConditions.visibilityOfElementLocated(balance)).getText();
    }

    public String getTotalBalance() {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(totalBalance)).getText();
    }
}

```

v. [TransferFundsPage.java](#)

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class TransferFundsPage {

    private WebDriver driver;
    private WebDriverWait wait;

    By transferLink = By.linkText("Transfer Funds");
    By amountField = By.id("amount");
    By fromAccountSelect = By.id("fromAccountId");
    By toAccountSelect = By.id("toAccountId");
    By transferButton = By.xpath("//input[@value='Transfer']");
    By successMessage = By.xpath("//h1[text()='Transfer Complete!']");
}

```

```

public TransferFundsPage(WebDriver driver) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, java.time.Duration.ofSeconds(10));
}

public void clickTansferFund() {
    wait.until(ExpectedConditions.elementToBeClickable(transferLink)).click();
}

public void transferFunds(String amount, String fromAcc, String toAcc) {

    wait.until(ExpectedConditions.visibilityOfElementLocated(amountField)).sendKeys(amount)
;

    Select fromDropdown = new Select(driver.findElement(fromAccountSelect));
    fromDropdown.selectByVisibleText(fromAcc);

    Select toDropdown = new Select(driver.findElement(toAccountSelect));
    toDropdown.selectByVisibleText(toAcc);
    driver.findElement(transferButton).click();
}

public String getSuccessMessage() {
    return wait.until(ExpectedConditions.visibilityOfElementLocated(successMessage)).getText();
}
}

```

vi. BillPayPage.java

```

package pages;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

```

```

import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class BillPayPage {

    private WebDriver driver;
    private WebDriverWait wait;

    By billPayLink = By.linkText("Bill Pay");
    By pageTitle = By.xpath("//h1[text()='Bill Payment Service']");
    By payeeName = By.name("payee.name");
    By payeeAddress = By.name("payee.address.street");
    By payeeCity = By.name("payee.address.city");
    By payeeState = By.name("payee.address.state");
    By payeeZipCode = By.name("payee.address.zipCode");
    By payeePhone = By.name("payee.phoneNumber");
    By payeeAccount = By.name("payee.accountNumber");
    By verifyPayeeAccount = By.name("verifyAccount");
    By amount = By.name("amount");
    By fromAccount = By.name("fromAccountId");
    By sendPaymentBtn = By.xpath("//input[@value='Send Payment']");
    By successMessage = By.xpath("//h1[text()='Bill Payment Complete']");

    public BillPayPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    public void navigateToBillPay() {
        wait.until(ExpectedConditions.elementToBeClickable(billPayLink)).click();
    }
}

```

```

public String getPageTitle() {
    wait.until(ExpectedConditions.visibilityOfElementLocated(pageTitle));
    return driver.findElement(pageTitle).getText();
}

public void fillBillPayForm(String name, String address, String city, String state, String
zipCode, String phone, String account, String verifyAccount, String amountValue, String
fromAccountId) {
    wait.until(ExpectedConditions.visibilityOfElementLocated(payeeName));
    driver.findElement(payeeName).sendKeys(name);
    driver.findElement(payeeAddress).sendKeys(address);
    driver.findElement(payeeCity).sendKeys(city);
    driver.findElement(payeeState).sendKeys(state);
    driver.findElement(payeeZipCode).sendKeys(zipCode);
    driver.findElement(payeePhone).sendKeys(phone);
    driver.findElement(payeeAccount).sendKeys(account);
    driver.findElement(verifyPayeeAccount).sendKeys(verifyAccount);
    driver.findElement(amount).sendKeys(amountValue);

    Select fromAccountDropdown = new
Select(wait.until(ExpectedConditions.visibilityOfElementLocated(fromAccount)));
    fromAccountDropdown.selectByVisibleText(fromAccountId);
}

public void submitPayment() {
    wait.until(ExpectedConditions.elementToBeClickable(sendPaymentBtn)).click();
}

public void waitForConfirmation() {
    wait.until(ExpectedConditions.visibilityOfElementLocated(successMessage));
}
}

```

vii. LogoutPage.java

```
package pages;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public class LogoutPage {
    private WebDriver driver;
    private WebDriverWait wait;

    By logoutLink = By.xpath("//a[text()='Log Out']");

    public LogoutPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    public void clickLogout() {
        try {
            wait.until(ExpectedConditions.elementToBeClickable(logoutLink)).click();
        } catch (Exception e) {
            System.out.println("Logout link not found. Assuming already logged out.");
        }
    }
}
```


III. src/test/java/testcases

i. RegistrationTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.RegistrationPage;
import utilities.ExcelUtils;

public class RegistrationTest extends BaseTest {

    @DataProvider(name="regData")
    public Object[][] getdata() throws Exception{
        return ExcelUtils.getTestData("src/test/resources/TestData.xlsx", "Sheet1");
    }

    @Test(dataProvider = "regData")
    public void testRegistration(String fname, String lname, String adr, String cty, String st,
String zp, String phn, String ssn, String user, String pwd, String cnfpwd) {
        RegistrationPage regPage = new RegistrationPage(driver);
        regPage.register(fname, lname, adr, cty, st, zp, phn, ssn, user, pwd, cnfpwd);

//        Validation
        String act_msg = regPage.confirmRegister();
        String exp_msg = "Welcome " + user;

        Assert.assertEquals(act_msg, exp_msg, "Registration Failed!");
        System.out.println("--- Registration Test Passed for: ---" + user);
    }
}
```

ii. LoginTest.java

```
package testcases;

import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import org.testng.Assert;
import pages.LoginPage;
import utilities.ExcelUtils;

public class LoginTest extends BaseTest{

    @DataProvider(name="loginData")
    public Object[][] getLoginData() throws Exception{
        return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet2");
    }

    @Test(dataProvider = "loginData", groups="LoginGroup")
    public void validLogin(String username, String password) {
        LoginPage loginpage = new LoginPage(driver);
        loginpage.loginUser(username, password);

//        Validation
        String exp_title = "ParaBank | Accounts Overview";
        String act_title = driver.getTitle();
        Assert.assertEquals(act_title, exp_title, "Login Failed!");
        System.out.println("--- Login Test Passed for: " + username + " ---");
    }

}
```

iii. NegativeLoginTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.LoginPage;
import utilities.ExcelUtils;

public class NegativeLoginTest extends BaseTest {

    @DataProvider(name="invalidLoginData")
    public Object[][] getInvalidData() throws Exception {
        // Create a new sheet 'Sheet3' in your Excel for negative test data
        return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet3");
    }

    @Test(dataProvider = "invalidLoginData")
    public void testNegativeLogin(String username, String password) {
        LoginPage loginPage = new LoginPage(driver);
        loginPage.loginUser(username, password);

        // Assert that the user is NOT logged in (look for error message or stay on login page)
        String currentUrl = driver.getCurrentUrl();

        // Assert.assertFalse(currentUrl.contains("overview"), "Login should have failed, but user was
        logged in!");

        Assert.assertTrue(loginPage.isLoginErrorDisplayed(), "Error message was not displayed!");
        System.out.println("--- Negative Login Test Passed (User not logged in) ---");
    }
}
```

iv. OpenAccountTest.java

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.LoginPage;
import pages.OpenAccountPage;
import utilities.ExcelUtils;

public class OpenAccountTest extends BaseTest {

    @DataProvider(name="loginData")
    public Object[][] getLoginData() throws Exception {
        return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet2");
    }

    @Test(dataProvider = "loginData", dependsOnGroups = "LoginGroup",
groups="OpenAccountGroup")
    public void testOpenNewAccount(String username, String password) {

        LoginPage loginPage = new LoginPage(driver);
        loginPage.loginUser(username, password);

        OpenAccountPage openAccPage = new OpenAccountPage(driver);

        openAccPage.clickOpenAccount();
        openAccPage.openNewAccount("SAVINGS");

//        Account Details
        BaseTest.newAccountId = openAccPage.getNewAccountId();

//        Output the details
        System.out.println("--- Open Account Test Passed for: ---" + username);
    }
}
```

```

        System.out.println("New Account ID: " + BaseTest.newAccountId);

        // Validation
        String exp_title = "ParaBank | Open Account";
        String act_title = driver.getTitle();
        Assert.assertEquals(act_title, exp_title, "Open Account Page not displayed!");
    }

}

```

v. AccountOverviewTest.java

```

package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.AccountOverviewPage;
import pages.LoginPage;
import utilities.ExcelUtils;

public class AccountOverviewTest extends BaseTest {

    @DataProvider(name="loginData")
    public Object[][] getLoginData() throws Exception {
        return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet2");
    }

    @Test(dataProvider = "loginData", dependsOnGroups = "LoginGroup",
groups="AccountOverviewGroup")
    public void testAccountOverviewDetails(String username, String password) {

```

```

// Login using the data provider
LoginPage loginPage = new LoginPage(driver);
loginPage.loginUser(username, password);

//Initialize Page
AccountOverviewPage accPage = new AccountOverviewPage(driver);

// Account details
BaseTest.capturedAccountId = accPage.getAccountId(1);

Assert.assertNotNull(BaseTest.capturedAccountId, "Primary Account ID not found on overview
page!");

// For second account
try {
    BaseTest.newAccountId = accPage.getAccountId(2);
} catch (Exception e) {
    System.out.println("Secondary account not found! Defaulting to primary account ID.");
    BaseTest.newAccountId = BaseTest.capturedAccountId;
}

String primaryBalance = accPage.getBalance(1);
String secondaryBalance = accPage.getBalance(2);
String total = accPage.getTotalBalance();

// Output the details
System.out.println("--- Account Overview for: " + username + " ---");
System.out.println("Primary Account: " + BaseTest.capturedAccountId);
System.out.println("Primary Account Balance: " + primaryBalance);
System.out.println("Secondary Account: " + BaseTest.newAccountId);
System.out.println("Secondary Account Balance: " + secondaryBalance);

System.out.println("Total: " + total);

// Validation

```

```

        Assert.assertNotNull(BaseTest.capturedAccountId);
        Assert.assertNotNull(BaseTest.newAccountId);
    }
}

```

vi. TransferFundsTest.java

```
package testcases;
```

```

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import base.BaseTest;
import pages.AccountOverviewPage;
import pages.LoginPage;
import pages.TransferFundsPage;
import utilities.ExcelUtils;

```

```
public class TransferFundsTest extends BaseTest {
```

```
    @DataProvider(name="transferData")
```

```
    public Object[][] getTransferData() throws Exception {
```

```
        return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet2");
```

```
    }
```

```

    @Test(dataProvider = "transferData", dependsOnGroups = "AccountOverviewGroup",
groups="TransferFundsGroup")

```

```
    public void testTransferFunds(String username, String password) {
```

```
        Assert.assertNotNull(BaseTest.capturedAccountId, "Source account is null!");
```

```
        Assert.assertNotNull(BaseTest.newAccountId, "Destination account is null!");
```

```
        LoginPage loginPage = new LoginPage(driver);
```

```

loginPage.loginUser(username, password);

// Check Current balances before transfer
AccountOverviewPage accPage = new AccountOverviewPage(driver);
String balStr = accPage.getBalance(1).replace("$", "").replace(",", "").trim();
double currentBal = Double.parseDouble(balStr);
double transferAmt = 50.00;

try {
    if(transferAmt > currentBal) {
        System.out.println("--- Insufficient funds for transfer! Current Balance: $" +
currentBal + " ---");
        Assert.fail("Insufficient funds for transfer!");
    }
} catch(Exception e) {
    System.out.println("--- Error during balance check: " + e.getMessage() + " ---");
    throw e; // Rethrow to fail the test
}

// Transfer
TransferFundsPage tfPage = new TransferFundsPage(driver);
tfPage.clickTansferFund();

tfPage.transferFunds(String.valueOf(transferAmt), BaseTest.capturedAccountId,
BaseTest.newAccountId);

// Validation
String actualMsg = tfPage.getSuccessMessage();
Assert.assertTrue(actualMsg.contains("Complete"), "Fund transfer failed!");

System.out.println("--- Fund Transfer Test Passed for user: " + username + " ---");
System.out.println("--- Fund Transfer Success for: " + BaseTest.capturedAccountId + " ---");

driver.findElement(By.linkText("Accounts Overview")).click();

```



```

String updatedPrimaryBal = accPage.getBalance(1);
String updatedSecondaryBal = accPage.getBalance(2);

System.out.println("--- Balance after Fund-Transfer ---");
System.out.println("Updated Primary Balance: " + updatedPrimaryBal);
System.out.println("Updated Secondary Balance: " + updatedSecondaryBal);

// Validation after fund transfer
Assert.assertTrue(updatedPrimaryBal.contains("$"), "Primary balance format invalid!");
    }
}

```

vii. BillPayTest.java

```

package testcases;

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.AccountOverviewPage;
import pages.BillPayPage;
import pages.LoginPage;
import utilities.ExcelUtils;
import utilities.ScreenshotUtil;

public class BillPayTest extends BaseTest {

    @DataProvider(name="loginData")

```

```

public Object[][] getLoginData() throws Exception {
    return ExcelUtils.getLoginData("src/test/resources/TestData.xlsx", "Sheet2");
}

@Test(dataProvider = "loginData", dependsOnGroups = "AccountOverviewGroup",
groups="BillPaymentGroup")
public void testBillPayment(String username, String password) {
    LoginPage loginPage = new LoginPage(driver);
    loginPage.loginUser(username, password);

//    Balance Check
    AccountOverviewPage accPage = new AccountOverviewPage(driver);
    String balStr = accPage.getBalance(1).replace("$", "").replace(",", "").trim();
    double currentBal = Double.parseDouble(balStr);
    double paymentAmt = 20.00;

//    Bill Pay
    BillPayPage billPayPage = new BillPayPage(driver);
    billPayPage.navigateToBillPay();

    Assert.assertEquals(billPayPage.getPageTitle(), "Bill Payment Service", "Failed to
navigate to Bill Pay page!");

    try {
        if (currentBal >= paymentAmt) {

            billPayPage.fillBillPayForm("testPayee", "321 Down Street", "Howrah",
"Bengal",
                "90210", "555-1234", "1234567890", "1234567890",
                String.valueOf(paymentAmt), BaseTest.capturedAccountId);
            billPayPage.submitPayment();

            billPayPage.waitForConfirmation();

            // Screenshot on Success

```

```

        ScreenshotUtil.takeScreenshot(driver, "BillPay_Success_" + username);
        System.out.println("--- Bill Payment Successful for: " + username + " ---");

//        Check Updated Balance
        driver.findElement(By.linkText("Accounts Overview")).click();
        String updatedBalStr = accPage.getBalance(1).replace("$", "").replace(", ",
"".trim());

        double updatedBal = Double.parseDouble(updatedBalStr);

        System.out.println("Current Balance: $" + currentBal + "\nPayment Amount:
$" + paymentAmt + "\nUpdated Balance: $" + updatedBal);

        System.out.println("-----");

        Assert.assertEquals(updatedBal, currentBal - paymentAmt, "Balance did not
update correctly after bill payment!");
    } else {
        throw new Exception("Insufficient balance. Required: " + paymentAmt + ",
Available: " + currentBal);
    }

} catch (Exception e) {
    System.out.println("--- Bill Pay Exception Handled: " + e.getMessage() + " ---");

// Screenshot on Failure/Exception
    ScreenshotUtil.takeScreenshot(driver, "BillPay_Failure_" + username);
}
}
}
}

```

viii. `LogOutTest.java`

```
package testcases;
```

```

import org.openqa.selenium.By;
import org.testng.Assert;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.LogoutPage;

public class LogoutTest extends BaseTest {

    @Test(dependsOnGroups = {"BillPaymentGroup", "TransferFundsGroup"}, groups =
"LogoutGroup")
    public void testLogout() {

        LogoutPage logoutPage = new LogoutPage(driver);
        logoutPage.clickLogout();

        // Validation: Ensure we are back on the login page (or main page)
        boolean isLoginButtonVisible = !driver.findElements(By.xpath("//input[@value='Log
In']")).isEmpty();

        Assert.assertTrue(isLoginButtonVisible, "Logout failed: Login button not found on
page!");
        System.out.println("--- Logout Successful ---");
    }
}

```

IV. src/test/java/utilities

i. ConfigReader.java

```
package utilities;

import java.io.FileInputStream;
import java.util.Properties;

public class ConfigReader {

    public static String getProperty(String key) throws Exception{

        Properties prop = new Properties();
        FileInputStream file = new FileInputStream("src/test/resources/config.properties");
        prop.load(file);
        return prop.getProperty(key);
    }
}
```

ii. ExcelUtils.java

```
package utilities;

import java.io.FileInputStream;

import org.apache.poi.sl.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelUtils {

    static XSSFWorkbook wb;

    static XSSFSheet ws;
```

```

public static Object[][] getTestData(String filepath, String xlsheet) throws Exception{
    FileInputStream file = new FileInputStream(filepath);
    wb = new XSSFWorkbook(file);
    ws = wb.getSheet(xlsheet);

    int rows = ws.getLastRowNum();
    int cols = ws.getRow(0).getLastCellNum();

    Object[][] data =new Object[rows][cols];

    for(int i=0;i<rows;i++)
    {
        for(int j=0;j<cols;j++)
        {
            data[i][j] = ws.getRow(i+1).getCell(j).toString();
        }
    }
    wb.close();
    return data;
}

```

```

public static Object[][] getLoginData(String filepath, String xlsheet) throws Exception{
    FileInputStream file = new FileInputStream(filepath);
    wb = new XSSFWorkbook(file);
    ws = wb.getSheet(xlsheet);

    int rows = ws.getLastRowNum();
    int cols = ws.getRow(0).getLastCellNum();

    Object[][] data =new Object[rows][cols];

    for(int i=0;i<rows;i++)
    {

```

```

        for(int j=0;j<cols;j++)
        {
            data[i][j] = ws.getRow(i+1).getCell(j).toString();
        }
    }
    wb.close();
    return data;
}
}

```

iii. ScreenshotUtil.java

package utilities;

```

import java.io.File;
import java.io.IOException;
import java.nio.file.Files; // Use this instead of FileUtils
import java.text.SimpleDateFormat;
import java.util.Date;

```

```

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

```

```

public class ScreenshotUtil {
    public static void takeScreenshot(WebDriver driver, String testName) {
        // Time stamp for the ss
        String timestamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss").format(new Date());

        String directoryPath = "./screenshots/";
        File directory = new File(directoryPath);
    }
}

```

```

// Create the folder if it doesn't exist
if (!directory.exists()) {
    directory.mkdirs();
}

// Create a unique filename for the screenshot
File destFile = new File(directoryPath + testName + "_" + timestamp + ".png");

// Take screenshot
File srcFile = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);

try {
    Files.copy(srcFile.toPath(), destFile.toPath());
    System.out.println("Screenshot saved: " + destFile.getAbsolutePath());
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

iv. TestListener.java

```
package utilities;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.testng.ITestListener;
```

```
import org.testng.ITestResult;
```

```
import base.BaseTest;
```

```
public class TestListener extends BaseTest implements ITestListener{
```



```

@Override

public void onTestFailure(ITestResult result) {

try {

ScreenshotUtil.takeScreenshot(BaseTest.getDriver(), result.getName());

System.out.println("Screenshot taken for failed test: " + result.getName());

} catch (Exception e) {

System.out.println("Failed to take screenshot: " + e.getMessage());

}

}

}

```

4. Src/test/resources

i. Config.properties

browser=[chrome](#)

appurl=<https://parabank.parasoft.com/parabank/index.htm>

ii. TestData.xlsx

• Sheet1

	A	B	C	D	E	F	G	H	I	J	K
1	FirstName	LastName	Address	City	State	Zip	Phone	SSN	User	Pass	CnfPass
2	TestNG 04	User 04	3/12 Main Street	Kolkata	Bengal	700060	5468792130	987-123	testnguser04	admin123	admin123
3											

• Sheet2

	A	B	C
1	User	Pass	
2	testnguser04	admin123	
3			
4			
5			

• Sheet 3

	A	B	C
1	User	Pass	
2	invaliduser	invalidpass	
3			
4			
5			
6			

< > ... Sheet2 **Sheet3** +

5. Testng.xml

```

<suite name="ParaBank Suite">
  <test name="Regression Tests">
    <classes>
      <class name="testcases.RegistrationTest"/>
      <class name="testcases.LoginTest"/>
      <class name="testcases.OpenAccountTest"/>
      <class name="testcases.AccountOverviewTest"/>
      <class name="testcases.TransferFundsTest"/>
      <class name="testcases.BillPayTest"/>
      <class name="testcases.LogoutTest"/>
      <class name="testcases.NegativeLoginTest"/>
    </classes>
  </test>
</suite>

```

6. Console Output

```
[RemoteTestNG] detected TestNG version 7.11.0
SLF4J(W): No SLF4J providers were found.
SLF4J(W): Defaulting to no-operation (NOP) logger implementation
SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.
2026-06-13T07:25:36.269117500Z main ERROR Log4j API could not find a logging provider.
Jun 13, 2026 12:55:38 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:55:38 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a de
--- Registration Test Passed for: ---testnguser04
Jun 13, 2026 12:55:46 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:55:46 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a de
--- Login Test Passed for: testnguser04 ---
Jun 13, 2026 12:55:50 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:55:50 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a de
--- Negative Login Test Passed (User not logged in) ---
Jun 13, 2026 12:55:54 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:55:54 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a de
--- Open Account Test Passed for: ---testnguser04
New Account ID: 16896

--- Account Overview for: testnguser04 ---
Primary Account: 16785
Primary Account Balance: $415.50
Secondary Account: 16896
Secondary Account Balance: $100.00
Total: $515.50
Jun 13, 2026 12:56:04 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:56:04 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a dependency on a specific version of the CDP using
--- Fund Transfer Test Passed for user: testnguser04 ---
--- Fund Transfer Success for: 16785 ---
--- Balance after Fund-Transfer ---
Updated Primary Balance: $365.50
Updated Secondary Balance: $150.00
Jun 13, 2026 12:56:11 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:56:11 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a dependency on a specific version of the CDP using
Screenshot saved: C:\Users\TAPABRATA\git\capstone_project\ParabankAutomation\.\screenshots\BillPay_Success_testnguser04_20260613_125617.png
--- Bill Payment Successful for: testnguser04 ---
Current Balance: $365.5
Payment Amount: $20.0
Updated Balance: $345.5
-----
Jun 13, 2026 12:56:19 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 130
Jun 13, 2026 12:56:19 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$3
WARNING: Unable to find version of CDP to use for 130.0.6723.117. You may need to include a dependency on a specific version of the CDP using
Logout link not found. Assuming already logged out.
--- Logout Successful ---

=====
ParaBank Suite
Total tests run: 8, Passes: 8, Failures: 0, Skips: 0
=====
```

Console Warnings — Clarification

During test execution, the console displays several WARNING-level messages in red. These are **framework-level informational notices** and are not indicative of any defect in the test code or the application under test.

The warning that appears:

- **CDP Version Mismatch Warning:** Selenium WebDriver logs a warning when it cannot find an exact Chrome DevTools Protocol (CDP) binding for the detected browser version (Chrome

130). Since this project does not utilise any CDP-dependent features such as network interception or browser-level event monitoring, this warning has zero impact on test behaviour or results.

These warnings are expected, documented, and confirmed to have no effect on the test suite. All 8 test cases passed successfully in every execution cycle, as evidenced by the final TestNG summary: *Total tests run: 8, Passes: 8, Failures: 0, Skips: 0.*

7. Jenkins file

```
pipeline {
    agent any

    tools {
        maven 'maven3'
        jdk 'java'
    }

    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out code from GitHub...'
            }
        }

        stage('Build & Test') {
            steps {
                // Navigate into the subdirectory where pom.xml is located
                dir('ParabankAutomation') {
                    // This runs your tests using the Maven installed on your local machine
                    bat 'mvn clean test'
                }
            }
        }
    }

    post {
        always {
            // Publish test results
            archiveArtifacts artifacts: 'ParabankAutomation/target/surefire-reports/*.xml', fingerprint: true
        }
    }
}
```

Jenkins

Parabank_CI

#17

Stages

Search

🔍

🔔

🟢

< #17

🔄 Rerun

⚙️

👤 Started by Tapabrata Sikder

🕒 Started 3 min 22 sec ago

🕒 Queued 7 ms

🕒 Took 1 min 42 sec

📦 Artifacts

Start

End

Checkout SCM

Tool Install

Checkout

Build & Test

Post Actions

🔍 Search

🔽

🟢 Checkout SCM 2s

🟢 Tool Install 0.18s

🟢 Checkout 0.31s

🟢 Build & Test 1m 36s

🟢 Post Actions 0.13s

🟢 Post Actions 0.13s

🕒 Started 9m 4s ago

Jenkins

🔗

🟢 Archive the artifacts

🔗 82ms

🔍 Archiving artifacts

1 Recording fingerprints